Meta-Reinforcement Learning-Based Adaptive Architecture for Large Language Model Multi-Agent Systems

Naveen Tanikonda
Vellore Institute Of Technology - Amaravathi
naveen.23bce9787@vitapstudent.ac.in

VeeraBhadra Jagarlamudi
Vellore Institute Of Technology - Amaravathi
bhadra.23bce9780@vitapstudent.ac.in

Sriram Somarouthu,
Vellore Institute Of Technology - Amaravathi
sriram.23bce20270@vitapstudent.sc.in

Ravi Teja Perumalapalli
Vellore Institute Of Technology - Amaravathi
teja.23bce9392@vitapstudent.ac.in

***Abstract*—Large Language Model (LLM)-based multi-agent systems facilitate complex task execution through collaboration among specialized agents. However, conventional systems often rely on fixed workflows, which can introduce unnecessary agent activations, communication overhead, increased execution latency, and higher inference costs across tasks with different requirements. This paper investigates an adaptive architecture for LLM-based multi-agent systems in which agent activation, role assignments, and communication relationships are modified according to task requirements [1]–[4]. The architecture selection problem is formulated as a Markov Decision Process (MDP), where reinforcement learning is used to select structural modifications while maintaining graph validity [5]–[7]. The proposed framework incorporates task-context encoding, value-based action selection, structural validation, and dynamic workflow construction using LangGraph. To investigate cross-task adaptation, a shared task-conditioned neural policy is trained across multiple task environments and evaluated on held-out tasks through limited task-specific updates [8]–[11]. Its adaptation performance is compared with that of a freshly initialized policy under the same update budget. The current implementation evaluates neural adaptation using a deterministic architecture–task compatibility measure rather than actual LLM-generated task outcomes. Preliminary prototype dashboard observations indicate lower token consumption and estimated execution cost for an adaptive configuration compared with a static configuration; however, these observations are not a controlled evaluation of the neural adaptation method. The study provides a foundation for investigating task-adaptive multi-agent architectures, with further experiments required to establish their effectiveness in actual LLM-based execution.**

‘‘‘

***Index Terms*—Large language models, multi-agent systems, reinforcement learning, meta-reinforcement learning, adaptive architecture, dynamic workflows, LangGraph.**

I. Introduction

Large Language Models (LLMs) have significantly expanded the capabilities of artificial intelligence systems in understanding natural language, generating content, solving problems, and performing complex reasoning tasks. However, handling complex tasks often requires more than a single LLM. Such tasks may involve multiple stages, including planning, information retrieval, reasoning, coding, and verification. To address these requirements, LLM-based Multi-Agent Systems (MAS) have emerged as an approach in which multiple specialized agents collaborate to complete tasks through structured interactions. Frameworks such as MetaGPT demonstrate how specialized agents can collaborate through role-based coordination and structured workflows [12]. Recent surveys have also discussed the architectures, applications, and challenges of LLM-based agents [13]–[15].

In a typical multi-agent system, individual agents are assigned specific responsibilities, and their interactions are coordinated through a predefined workflow. Although this approach enables task decomposition and collaboration, conventional architectures frequently rely on fixed agent configurations and communication structures. Consequently, the same architecture may be used for tasks with substantially different requirements. For example, a simple question-answering task may require only one agent, whereas a complex research task may benefit from multiple specialized agents. Using an unnecessarily large agent configuration can introduce redundant processing, additional communication, increased token consumption, and higher execution costs. Recent research has investigated these challenges through adaptive agent selection, communication pruning, and dynamic topology generation [1]–[3].

An important challenge in LLM-based multi-agent systems is therefore determining how the architecture should change according to the requirements of a given task. An adaptive architecture should be capable of selecting relevant agents, assigning appropriate roles, and modifying communication relationships while preserving the validity of the workflow. Such adaptation can potentially reduce unnecessary agent execution and improve resource utilization without removing the capabilities required to complete a task. Approaches such as Adaptive Graph Pruning investigate the joint optimization of agent quantity and communication topology, while

AgentDropout explores the dynamic deactivation of redundant agents during collaboration [1], [3]. Similarly, graph-based topology generation has been explored to construct communication structures according to task requirements [2].

Reinforcement Learning (RL) provides a possible approach to this problem by treating architecture selection as a sequential decision-making process. An RL agent can observe task-related information and the current multi-agent configuration, select structural actions, and receive rewards based on the resulting architecture. Previous studies have investigated reinforcement-learning-based coordination and collaboration among multiple agents, including LLM-based systems [5], [6], [16]. In this work, the architecture adaptation problem is formulated as a Markov Decision Process (MDP). The action space includes operations such as activating or deactivating agents, modifying agent roles, and updating communication relationships. Structural validation is incorporated to ensure that the resulting architecture satisfies the required workflow constraints.

However, an architecture that performs well on one task may not necessarily be suitable for another. This motivates the investigation of cross-task learning, in which information acquired from multiple task environments can support adaptation to previously unseen tasks. Meta-reinforcement learning and multi-task reinforcement learning have been investigated as approaches for transferring useful knowledge across tasks and supporting adaptation to new environments [8]–[11]. The proposed framework explores this direction through a shared task-conditioned neural policy trained across multiple task environments. The trained policy is subsequently evaluated on held-out tasks using a limited number of task-specific updates and compared with a freshly initialized policy under the same update budget. This experimental approach investigates whether knowledge learned across tasks can provide a useful starting point for adaptation. It should be noted that the implemented approach is a task-conditioned neural adaptation method and does not implement a canonical Meta-RL algorithm such as MAML, Reptile, or PEARL.

The proposed architecture also uses LangGraph to represent and construct dynamic multi-agent workflows. Task information is used to guide architecture selection, while structural validation helps maintain valid agent connections and execution paths. The framework separates the decision-making process from workflow execution, allowing the selected architecture to be translated into an executable multi-agent structure. Related research on adaptive multi-agent architecture search and learned execution policies provides additional motivation for investigating dynamic workflow construction [17], [18].

The main contributions of this work are summarized as follows:

- **Adaptive Multi-Agent Architecture:** A task-dependent architecture adaptation framework is developed to modify agent activation, role assignments, and communication relationships according to task requirements.
- **MDP-Based Architecture Selection:** The architecture adaptation problem is formulated as a Markov Decision Process, enabling reinforcement-learning-based selection of structural modifications while maintaining workflow validity.
- **Cross-Task Neural Adaptation:** A shared task-conditioned neural policy is trained across multiple task environments and evaluated on held-out tasks through limited task-specific updates. Its performance is compared with a freshly initialized policy under an equivalent update budget.
- **Dynamic Workflow Construction:** LangGraph is integrated to represent and construct multi-agent workflows based on the selected architecture.
- **Preliminary Evaluation:** A prototype dashboard and a supplementary mock benchmark are used to examine architecture-related metrics. The evaluation distinguishes preliminary dashboard observations from controlled experiments required to establish the effectiveness of cross-task neural adaptation.

The remainder of this paper is organized as follows. Section II discusses related work in LLM-based multi-agent systems, reinforcement learning, and adaptive architectures. Section III defines the problem and research objectives. Section IV presents the proposed system architecture, followed by the MDP formulation in Section V. Section VI describes the implementation details. Section VII presents the experimental methodology and preliminary results. Section VIII discusses the limitations and challenges, and Section IX concludes the paper and outlines future research directions. “‘

II. Related Work

A. LLM-Based Multi-Agent Systems

LLM-based Multi-Agent Systems (MAS) coordinate multiple language-model-based agents with distinct roles to solve complex tasks through collaboration. Frameworks such as MetaGPT demonstrate role-based task decomposition and structured communication among specialized agents [12]. Recent research has further investigated collaborative decision-making, reinforcement-learning-based coordination, and evolving orchestration strategies for LLM-based agents [5], [6], [16], [19]. Surveys of LLM-based agents and LLM-enhanced reinforcement learning also discuss their architectures, applications, and limitations [13]–[15].

Although these systems demonstrate the benefits of agent specialization and collaboration, selecting an appropriate number of agents and determining their interactions remain important challenges. A fixed workflow may activate unnecessary agents when a task requires only a limited set of capabilities. Recent approaches address this issue through dynamic agent selection and redundancy elimination. For example, AgentDropout investigates dynamically deactivating redundant agents to improve collaboration efficiency [3]. These studies motivate the development of multi-agent architectures that can adjust their structure according to task requirements.

B. Workflow and Graph-Based Orchestration

Graph-based orchestration represents a multi-agent workflow through nodes and directed edges. Nodes represent agents or operations, while edges define communication paths and possible execution transitions. This representation provides a structured way to organize agent interactions and construct workflows for different tasks.

Recent research has investigated the automatic generation and optimization of multi-agent communication structures. Adaptive Graph Pruning explores the optimization of agent quantity and communication topology [1]. Similarly, graph diffusion models have been investigated for dynamically generating communication topologies among multiple LLM agents [2]. Multi-agent architecture search has also been explored through agentic supernet approaches [17]. In addition, NeuralFSM investigates adaptive coordination through learned finite-state execution policies [18].

These approaches demonstrate the importance of selecting suitable agent configurations and communication structures. In the proposed framework, LangGraph is used to represent and construct the selected workflow. The architecture adaptation process determines the required agents and communication relationships, while structural validation is used to maintain valid workflow connectivity before execution.

C. Reinforcement Learning for Architecture Selection

Reinforcement Learning (RL) provides a framework for sequential decision-making in which an agent observes an environment state, selects an action, and receives a reward based on the resulting transition. Recent research has investigated the integration of RL with LLM-based multi-agent systems for collaboration, decision-making, and performance optimization [5], [6], [16]. Research on LLM-based multi-agent reinforcement learning has also explored the opportunities and challenges of combining language-model capabilities with multi-agent learning methods [14].

In adaptive multi-agent systems, RL can be applied to architecture selection by treating the workflow configuration as part of the environment state. The available actions may include activating or deactivating agents, modifying agent roles, and adding or removing communication links. The reward function can consider factors such as task suitability, architecture complexity, and execution overhead.

Recent approaches provide related directions for this problem. Agent Q-Mix investigates reinforcement-learning-based action selection for LLM-based multi-agent systems [7]. Adaptive Agentic Meta-Controller explores deep reinforcement learning for intelligent LLM and SLM orchestration [20]. These studies motivate the use of learned decision-making mechanisms for selecting and coordinating agents. The present work focuses specifically on modifying the structure of a multi-agent workflow through a sequence of architecture-level actions.

D. Task-Conditioned Reinforcement Learning

Task-conditioned reinforcement learning incorporates information about the current task into the state representation or policy input. The policy uses task-related features together with the current environment state to select actions that maximize the expected cumulative reward. This allows a policy to select different actions depending on the requirements of different tasks.

For adaptive multi-agent systems, task-conditioned reinforcement learning can use task complexity, required capabilities, and architectural state to determine which agents should be activated and how they should communicate. For example, a mathematical task may require a planner and calculator, whereas a coding task may require a planner, coder, and critic. Research on adaptive multi-agent coordination has investigated the use of task context to support coordination across tasks with different team requirements [8]. Other work has explored adaptive domain modeling and multi-agent task planning using language models [21].

Although task conditioning supports task-dependent action selection, it does not by itself establish meta-learning. A task-conditioned policy may learn a mapping between task features and architecture actions without explicitly optimizing a learning procedure for rapid adaptation to new task environments. Therefore, evaluating a policy on different task instances does not necessarily demonstrate adaptation to an unseen task distribution.

E. Meta-Reinforcement Learning

Meta-Reinforcement Learning (Meta-RL) focuses on learning how to adapt reinforcement-learning policies across a distribution of tasks. Instead of optimizing a policy for only one environment, meta-learning approaches use experience from multiple tasks to improve subsequent task-specific adaptation. Research has investigated contextual meta-reinforcement learning for adaptive multi-agent coordination [8], as well as meta-reinforcement learning for emergent communication and zero-shot coordination among agents [10].

A general Meta-RL framework involves two stages: meta-training and meta-testing. During meta-training, the learning procedure uses experience from multiple training tasks to optimize shared parameters or an adaptation mechanism. During meta-testing, the learned parameters or adaptation procedure are evaluated on held-out tasks to determine whether previous experience supports faster or more effective adaptation. Related research has also explored knowledge transfer across multiple reinforcement-learning tasks [9] and fast policy adaptation through mixtures of orthogonal experts [11].

In LLM-based multi-agent systems, cross-task learning may help identify reusable architectural patterns. For example, experience from mathematical reasoning, coding, and research tasks may help a policy identify useful agent combinations when adapting to another task. However, task-conditioned action selection and Meta-RL represent different learning objectives. Task conditioning provides information about the

current task, whereas Meta-RL additionally optimizes the ability to adapt using experience acquired across tasks.

The present work investigates cross-task neural adaptation through a shared task-conditioned policy trained across multiple task environments. The policy is evaluated on held-out tasks using a limited number of task-specific updates and compared with a freshly initialized policy under the same update budget. This approach is intended to examine whether cross-task training provides a useful starting point for adaptation. It should not be interpreted as an implementation of a canonical Meta-RL algorithm such as MAML, Reptile, or PEARL.

TABLE I
COMPARISON OF TASK-CONDITIONED RL AND META-RL

Aspect	**Task-Conditioned RL**	**Meta-RL**
Learning objective	Learn task-dependent action selection	Learn an effective adaptation strategy across tasks
Task information	Current task features	Experience from a distribution of tasks
Training	Policy optimization using task information	Meta-training across multiple task environments
Unseen tasks	Generalization depends on the learned policy	Adaptation is explicitly evaluated on held-out tasks
Adaptation	Uses the trained policy for action selection	Uses a learned initialization or adaptation procedure
Evaluation	Performance across task instances	Adaptation performance on held-out tasks

F. Research Gap

Existing research has investigated LLM-based agent collaboration, adaptive agent selection, communication topology optimization, and reinforcement-learning-based coordination. Approaches such as Adaptive Graph Pruning, AgentDropout, and graph-based topology generation demonstrate the importance of modifying multi-agent structures according to task requirements [1]–[3]. Other studies investigate architecture search and learned execution policies for multi-agent coordination [17], [18].

However, selecting an architecture using current task features is different from learning a policy that can reuse experience across multiple task environments. This distinction motivates the investigation of cross-task neural adaptation for multi-agent architecture selection.

This work addresses the problem by formulating architecture modification as a sequential decision-making process and investigating a shared task-conditioned neural policy trained across multiple task environments. The framework considers agent activation, role assignment, and communication relationships as architecture-level decisions, while structural constraints are used to preserve workflow validity. Its effectiveness needs to be established through controlled comparisons with task-conditioned reinforcement learning and freshly initialized policies, together with evaluation on held-out task environments. “‘

III. PROBLEM DEFINITION AND OBJECTIVES

Let a multi-agent workflow be represented as a directed graph

$$G = (V, E), \tag{1}$$

where V is the set of available agent nodes and E is the set of directed communication or control edges. At a given time, only a subset of agents may be active. Each active agent may also have an assigned role or capability.

A fixed workflow uses the same graph for every request, regardless of task complexity or required capabilities. This can lead to unnecessary execution for simple tasks and insufficient flexibility for tasks that require additional capabilities during execution. Adaptive agent selection and communication-structure optimization have been investigated to address these challenges [1]–[3].

The objective is to select a valid architecture G_t for a task context τ_t so that task requirements are covered while unnecessary structural and execution overhead is reduced. The selection must satisfy graph constraints and preserve the ability to execute the workflow.

The framework is designed around the following objectives:

- select agents according to task capabilities and estimated complexity;
- reduce unnecessary agent invocations and communication edges;
- preserve graph validity through invariant checks;
- support conditional routing and quality-based rework;
- allow reassessment when execution reveals new requirements;
- collect evaluation signals for future policy improvement.

A. Task Distribution

Let $\mathcal{T}$ represent a distribution of task environments. Each task environment $\tau_i \sim \mathcal{T}$ represents a particular category of user request with its own capability requirements, complexity, and execution characteristics.

The task distribution may include mathematical reasoning, coding, research, web search, and cross-domain tasks. Each task is represented using a task-context vector containing information such as estimated complexity, required capabilities, tool requirements, and task type. Task-dependent coordination and contextual adaptation have been explored in multi-agent reinforcement learning [8], [9].

The objective is to learn an architecture adaptation policy that can use experience across these task environments to select suitable multi-agent configurations.

For experimental evaluation, the task distribution is divided into training, validation, and testing subsets. Training tasks are used during meta-training, validation tasks are used for model selection and hyperparameter tuning, and held-out testing tasks are reserved for meta-testing. Cross-task learning and

knowledge transfer provide related approaches for reusing experience across multiple task environments [9]–[11].

B. Difference Between Task-Conditioned Learning and Meta-Learning

In ordinary task-conditioned reinforcement learning, the architecture selection policy receives the current task context and architectural state to select an action. Its policy can be represented as:

$$a_t \sim \pi_\theta(a_t \mid s_t, c_t)$$

where s_t represents the architectural state and c_t represents the current task context.

In the proposed Meta-RL formulation, the policy is parameterized by meta-parameters θ. These parameters are learned across multiple task environments. For a particular task τ_i, the policy is adapted using task-specific experience D_i:

$$\theta_i' = \mathrm{Adapt}(\theta, D_i)$$

The adapted policy is then used to select architecture modifications for that task. Contextual meta-reinforcement learning and cross-task policy adaptation have been investigated in previous multi-agent learning studies [8], [10], [11].

Thus, task conditioning provides information about the current task, whereas meta-learning uses experience across multiple tasks to improve the adaptation process.

C. Research Objectives

The proposed framework focuses on the following objectives:

- To represent multi-agent workflows as directed graphs containing active agents, roles, and communication relationships.
- To formulate architecture modification as a reinforcement learning problem with a constrained discrete action space.
- To learn reusable adaptation knowledge from multiple task environments through meta-training.
- To evaluate whether the learned adaptation mechanism can improve architecture selection on held-out tasks during meta-testing.
- To reduce unnecessary agent execution and communication overhead while maintaining response quality.
- To evaluate the trade-off between task success, token consumption, execution latency, estimated cost, and architecture complexity.

‘‘‘

IV. System Architecture

Fig. 1 illustrates the overall architecture of the proposed Adaptive MAS framework. The framework processes user queries through task analysis and Meta-RL-based architecture selection. The selected architecture is validated and compiled into a dynamic LangGraph workflow, which coordinates the execution of specialized agents. Runtime reassessment enables the system to modify its architecture when task requirements change. Finally, evaluation feedback is used to update the reinforcement learning policy [1], [2], [5].

A. Ingestion, Memory, and Planning

The ingestion component receives a user query and associates it with the relevant conversation thread. The thread message store maintains conversation context so that subsequent requests can be interpreted with prior messages. The runtime orchestrator coordinates the request lifecycle and passes the query to the planner.

The planner produces a structured task description. The description may include required capabilities, tool requirements, and an estimated complexity level. These fields provide task information for architecture selection. The exact schema and values are implementation-dependent.

B. Architecture State and Policy Selection

The architecture state encoder represents the current workflow configuration. The task context encoder represents information about the request. These representations are supplied to the policy selection component. During exploitation, the policy uses learned action values to select a candidate action. During exploration, an LLM-assisted adapter may propose a candidate architecture change. The candidate is not accepted solely because it was proposed; it must pass the architecture manager's validation checks [1], [2].

C. Meta-Learned Architecture Adaptation

The meta-learning component provides a mechanism for reusing architectural experience collected from multiple task environments. It receives task-related information from the task-context encoder and uses the learned meta-parameters to initialize or adapt the architecture selection policy [8], [10], [11].

During meta-training, the adaptation mechanism is optimized using experience collected from different training task environments. This experience includes architecture states, selected actions, execution outcomes, reward signals, and resource consumption [9].

When a new task is received, the meta-learned parameters provide the starting point for task-specific adaptation. The architecture selection policy then evaluates the current task context and architectural state to determine appropriate modifications.

The selected actions are passed to the Architecture Manager, which checks structural constraints before the updated architecture is compiled into a dynamic LangGraph workflow [1], [2].

Following execution, the evaluation component collects response quality, capability coverage, token usage, latency, and architecture-related feedback. These signals provide task-specific experience for policy adaptation and subsequent learning.

The meta-learning component and the task-conditioned architecture selector therefore have distinct roles. The meta-learning component improves the adaptation mechanism

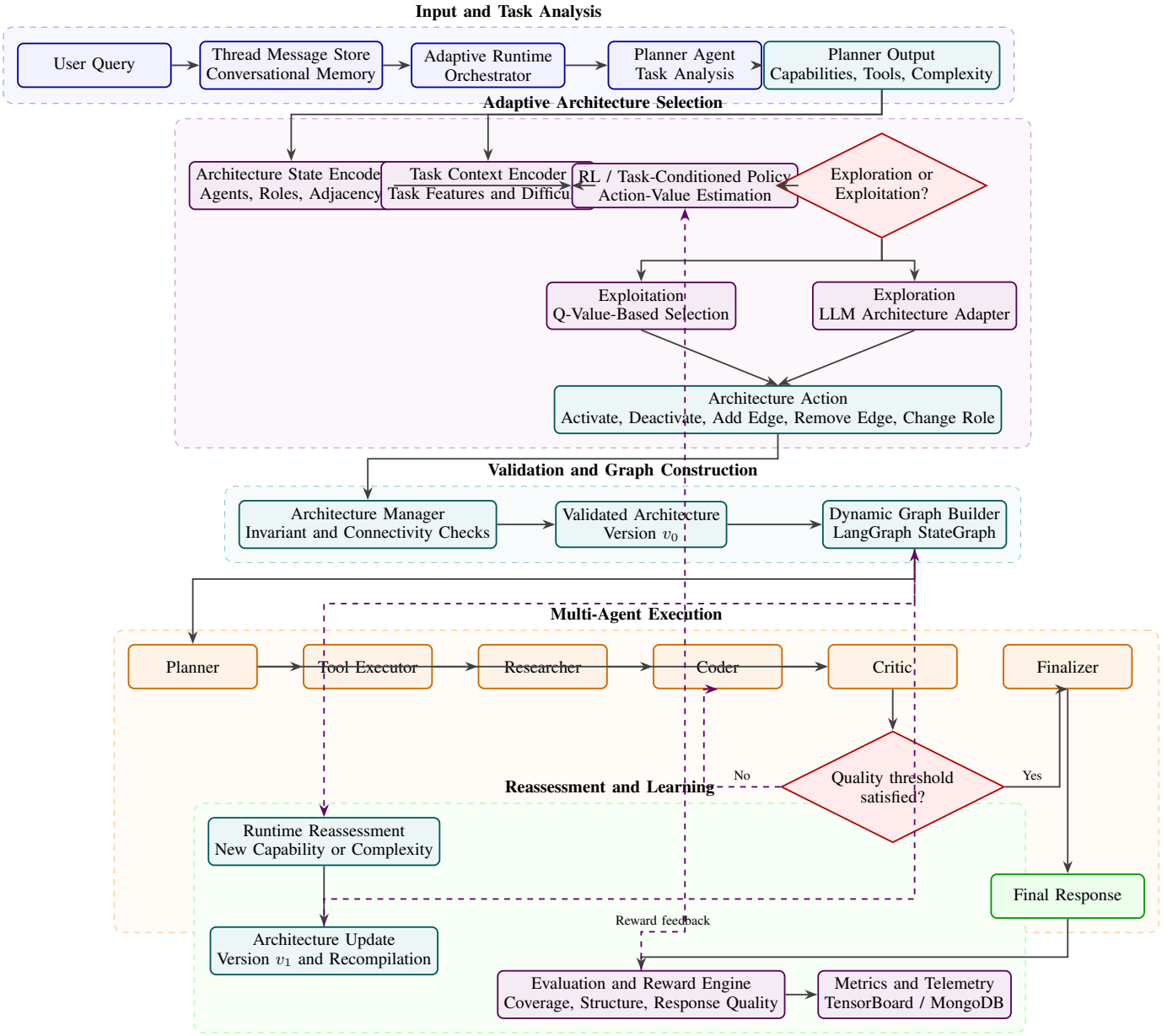


Fig. 1. Conceptual architecture of the Meta-RL-based Adaptive MAS framework. Conditional paths and feedback connections represent the intended runtime capabilities; their execution depends on the selected entrypoint and task configuration.

across tasks, while the architecture selector uses the resulting policy to make architecture modification decisions.

D. Meta-Training and Meta-Testing Workflow

The proposed framework follows two main learning stages.

Meta-Training: During meta-training, the policy is exposed to a distribution of training tasks. Each task environment provides experience through architecture selection, multi-agent execution, and reward evaluation. The meta-learning procedure uses this experience to update the shared meta-parameters so that the policy can adapt effectively across different task environments [8]–[10].

Meta-Testing: During meta-testing, the learned meta-parameters are evaluated on task instances or task distributions excluded from meta-training. The policy uses its learned adaptation mechanism and permitted task-specific experience to select an architecture. The resulting performance is measured in terms of adaptation effectiveness, response quality, resource consumption, and execution efficiency.

The testing procedure must prevent information from held-out tasks from influencing meta-training or validation. This separation is necessary to evaluate whether the learned mechanism can adapt beyond the training task distribution.

E. Architecture Validation and Dynamic Graph Construction

The architecture manager applies structural constraints to candidate changes. These constraints are intended to prevent invalid configurations, such as disconnected required execution paths or unsupported mutations. A validated architecture is then passed to the dynamic graph builder, which compiles the selected nodes and edges into a LangGraph state graph [1], [2], [18].

F. Multi-Agent Execution

The available agent pool includes a planner, tool executor, researcher, coder, critic, and finalizer. The selected graph determines which agents are active and how control can pass between them. The tool executor can invoke supported tools, including calculation, web search, academic search, and date/time utilities, subject to the configured tool environment [12], [19].

A critic or quality-evaluation stage can assess an intermediate result. When the configured quality condition is not satisfied, the workflow may route the task for correction. The project description documents a quality threshold of 0.80 and a bounded correction process; the exact path must be checked against the runtime configuration used for an experiment.

G. Runtime Reassessment

Initial planning may not identify every requirement. Runtime reassessment provides a mechanism to detect a missing capability or a change in task complexity. If reassessment is triggered, the architecture can be updated and the graph rebuilt. This mechanism is intended to avoid treating the initial graph as immutable [2], [18], [22].

H. Evaluation and Telemetry

The evaluation component collects architectural and response-level signals. Architectural signals can include capability coverage and structural compactness. Response-level signals can include a quality score or user feedback. Metrics are recorded through the configured telemetry and persistence components. The resulting reward signal can be used by the RL policy during subsequent learning or selection. “‘ “‘latex

V. MDP Formulation for Architecture Adaptation

The adaptive architecture selection problem is formulated as a Markov Decision Process (MDP), represented by

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma), \tag{2}$$

where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $\mathcal{P}$ represents the state transition function, $\mathcal{R}$ denotes the reward function, and $\gamma \in [0, 1]$ is the discount factor.

The objective is to select an appropriate multi-agent architecture according to the requirements of an input task while maintaining structural validity and minimizing unnecessary agent interactions [1]–[3]. The architecture selection process considers the current agent configuration, assigned roles, communication relationships, and task-specific context [8], [23].

The implementation contains two learning approaches: a tabular Q-learning baseline and a task-conditioned neural policy. The neural policy is trained across multiple tasks using shared parameters, followed by experiments that investigate adaptation to held-out tasks [9]–[11]. These approaches are described separately below.

A. State Representation

The state represents the current multi-agent architecture and the context of the input task. It is defined as

$$s_t = \langle v_t, r_t, A_t, c_t \rangle, \tag{3}$$

where v_t denotes the active-agent indicator vector, r_t represents the role assignments, A_t is the directed adjacency matrix, and c_t represents the task context [1], [2], [4].

For an architecture containing n available agents, the active-agent vector and adjacency matrix are represented as

$$v_t \in \{0, 1\}^n, \qquad A_t \in \{0, 1\}^{n \times n}. \tag{4}$$

The task context contains information such as task category, difficulty, required capabilities, and additional numerical features. The neural policy uses a deterministic task encoder to convert these features into a fixed-dimensional vector. Similarly, the architecture-state encoder represents agent activity, role assignments, adjacency relationships, and agent-count information as a numerical vector [8], [9].

The resulting state and task-context representations are provided as inputs to the task-conditioned neural policy.

B. Action Space

The action space defines the permitted modifications to the multi-agent architecture. It consists of the following discrete actions:

$$\begin{aligned} \mathcal{A} = \{ & \mathrm{ACTIVATE}(i), \mathrm{DEACTIVATE}(i), \\ & \mathrm{ADD_EDGE}(i, j), \mathrm{REMOVE_EDGE}(i, j), \\ & \mathrm{CHANGE_ROLE}(i, r) \}. \end{aligned} \tag{5}$$

Here, i and j identify agent nodes, while r represents the new role assigned to an agent. The action mapper converts between discrete action identifiers and architecture modification operations [1]–[4].

Only actions permitted by the architecture environment are considered valid. This restriction prevents the selection of unsupported architecture modifications.

C. State Transition and Structural Validation

When an action a_t is selected in state s_t, the environment generates a candidate next state:

$$\widetilde{s}_{t+1} = T(s_t, a_t). \tag{6}$$

The architecture manager validates the proposed modification before accepting the transition. The accepted state is defined as

$$s_{t+1} = \begin{cases} \widetilde{s}_{t+1}, & \text{if the transition is valid,} \\ s_t, & \text{otherwise.} \end{cases} \tag{7}$$

An invalid transition is rejected and receives a negative reward in the neural training procedure. This validation mechanism separates policy-based action selection from the structural modification of the architecture [1], [2], [18].

D. Reward Function

The reward function evaluates the quality of an architecture modification according to task performance and structural constraints. A general representation is

$$R_t = w_c C_{\text{cov}} + w_s C_{\text{struct}} + B_{\text{simp}} - P_{\text{bloat}} + w_q Q_{\text{resp}} + H_{\text{user}}, \tag{8}$$

where C_{cov} measures capability coverage, C_{struct} represents structural efficiency, B_{simp} denotes the simplification bonus, P_{bloat} penalizes unnecessary architectural components, Q_{resp} represents response quality, and H_{user} represents optional user feedback. The coefficients determine the relative contribution of these components [1], [3], [11].

In the neural adaptation experiment, the reward combines the structural reward provided by the environment with the change in task-performance score:

$$R_t^{\text{neural}} = R_t^{\text{struct}} + \left(Q_{\text{task}}(s_{t+1}) - Q_{\text{task}}(s_t)\right). \tag{9}$$

If the architecture modification is invalid, the training procedure assigns a reward of -1.0. The task-performance score is obtained using a deterministic architecture-task compatibility evaluator. Therefore, this score represents proxy evaluation rather than the result of executing an actual LLM-based multi-agent task.

E. Tabular Q-Learning Baseline

Tabular Q-learning is implemented as a baseline for learning architecture-selection decisions. The action-value function estimates the expected cumulative reward for selecting an action in a given state [23], [24].

The Q-value update is expressed as

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[R_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big], \tag{10}$$

where α is the learning rate and γ is the discount factor.

An ϵ-greedy strategy can be used to balance exploration and exploitation. During exploration, an action is selected from the available candidates, whereas during exploitation, the action with the highest estimated Q-value is selected.

This tabular implementation serves as a separate baseline and should not be interpreted as the neural cross-task adaptation method.

F. Task-Conditioned Neural Policy

The task-conditioned neural policy estimates an action score using both the architecture state and the task context. Let x_t denote the encoded architecture state and z_t denote the encoded task context. The policy is represented as

$$\mathbf{q}_\theta(s_t, c_t) = f_\theta([x_t \| z_t]), \tag{11}$$

where $[\cdot\|\cdot]$ denotes vector concatenation, f_θ is a feed-forward neural network, and $\mathbf{q}_\theta$ contains one score for each available architecture action [8], [9], [11].

The implemented network contains two hidden layers with ReLU activation functions and an output layer that produces action scores. The selected action is obtained by choosing the highest-scoring valid action:

$$a_t = \arg\max_{a \in \mathcal{A}_{\text{valid}}} q_\theta(s_t, c_t, a). \tag{12}$$

The neural policy is updated using a one-step temporal-difference target:

$$y_t = R_t^{\text{neural}} + \gamma(1 - d_t) \max_{a' \in \mathcal{A}_{\text{valid}}} q_\theta(s_{t+1}, c_t, a'), \tag{13}$$

where d_t indicates whether the episode has terminated. When the episode terminates or no valid next action exists, the bootstrap contribution is zero.

The selected action score is trained toward this target using mean squared error:

$$\mathcal{L}(\theta) = \left(q_\theta(s_t, c_t, a_t) - y_t\right)^2. \tag{14}$$

The network parameters are updated using the Adam optimizer to minimize this loss. This update describes the implemented task-conditioned neural policy and is distinct from the tabular Q-learning update.

G. Cross-Task Training and Fast Adaptation

The neural training procedure uses a shared task-conditioned policy across multiple training tasks. Let $\mathcal{T}_{\text{train}}$ denote the training task set and $\mathcal{T}_{\text{test}}$ denote the held-out task set, with no overlapping task identifiers [9]–[11].

During cross-task training, the shared policy is updated using task-specific architecture states, task contexts, selected actions, and observed rewards. The parameters are shared across the training tasks rather than maintaining an independent neural policy for each task [8], [9].

For an unseen task, the trained policy is evaluated before and after a limited number of task-specific updates. Its adaptation performance is also compared with a freshly initialized policy trained using the same update budget [10], [11].

The adaptation improvement is calculated as

$$\Delta Q_{\text{adapt}} = Q_{\text{after}} - Q_{\text{before}}, \tag{15}$$

where Q_{before} and Q_{after} represent the proxy task-performance scores before and after adaptation, respectively.

The advantage over training from scratch is measured as

$$\Delta Q_{\text{transfer}} = Q_{\text{adapted}} - Q_{\text{scratch}}, \tag{16}$$

where Q_{adapted} is the performance of the shared-policy model after adaptation and Q_{scratch} is the performance of a freshly initialized policy after the same number of updates.

These experiments investigate whether shared training across tasks provides an advantage during adaptation to held-out tasks. The evaluation uses a deterministic architecture-task compatibility proxy and does not, by itself, establish improved performance in actual LLM-based multi-agent execution. “‘

VI. Implementation

The proposed adaptive multi-agent system is implemented using a modular architecture that separates agent management, architecture modification, reinforcement learning, workflow execution, and performance evaluation. The implementation supports dynamic architecture selection based on task requirements and provides separate learning components for tabular Q-learning and task-conditioned neural policy learning [1], [2], [17].

The main implementation components are organized into the following modules:

- `app/memory/`: Manages conversation and thread memory.
- `app/architecture/`: Defines architecture models, modification operations, structural validation, and architecture versioning.
- `app/agents/`: Contains the specialized multi-agent components.
- `app/tools/`: Provides interfaces for external tools and their execution.
- `app/graph/`: Handles dynamic workflow construction and integration.
- `app/rl/`: Implements state representation, action mapping, policy selection, and learning procedures.
- `app/runtime/`: Manages request execution and runtime reassessment.
- `app/evaluation/`: Contains reward calculation, task-performance evaluation, and metric tracking.

A. Agent Pool and Responsibilities

The system uses a collection of specialized agents to perform different responsibilities during task execution. The agent pool contains the following components [12], [13], [19]:

- **Planner:** Interprets the incoming request, identifies task requirements, and determines the capabilities required for execution.
- **Researcher:** Collects relevant information using configured search and retrieval tools.
- **Coder:** Generates or modifies source code for programming-related tasks.
- **Tool Executor:** Invokes supported external tools to perform operations such as calculations and information retrieval.
- **Critic:** Evaluates intermediate outputs and identifies issues that may require correction or revision.
- **Finalizer:** Combines the outputs produced by the active agents and generates the final response.

The architecture manager determines the active agents and their communication relationships. Consequently, the system can construct different execution paths according to the capabilities required by the input task [1], [2], [18].

B. Architecture Management and Action Selection

The architecture management component represents the multi-agent workflow using agent nodes, role assignments, and communication relationships. Architecture modifications are represented through discrete actions supported by the architecture action mapper [1], [2].

The action mapper converts between discrete action identifiers and architecture modification operations. These operations include agent activation, agent deactivation, role modification, and communication-edge addition or removal [3], [4].

The `active_selector.py` module implements an epsilon-greedy Q-learning-based architecture selection mechanism. During exploitation, the selector uses learned Q-values to identify a suitable action. During exploration, it can obtain candidate architecture modifications from the LLM-based architecture adapter. When the adapter is unavailable, the selector can use random action exploration [5]–[7].

The selected modifications are passed to the architecture manager for validation. Only accepted modifications are used to construct the updated architecture. This separates the learning-based selection process from structural validation and workflow construction [1], [2], [18].

C. Tabular Q-Learning Implementation

The `q_learning.py` module implements a tabular Q-learning policy that serves as a baseline for architecture selection. The policy maintains action-value estimates for architecture states and updates them using observed rewards and subsequent state values [23], [24].

The implementation uses the standard temporal-difference Q-learning update to improve action-value estimates. The learned values are used by the active architecture selector to balance exploration and exploitation.

The repository also contains a multi-task training component in `meta_trainer.py`, which supports learning architecture-selection values across different task contexts. This component is distinct from the task-conditioned neural training procedure described below.

D. Task-Conditioned Neural Policy

The `task_conditioned_policy.py` module implements a neural policy that estimates architecture-action scores using both the current architecture state and task context [8], [9].

The architecture-state encoder represents agent activity, role assignments, adjacency information, and agent-count features as a numerical vector. The deterministic task encoder represents task category, difficulty, required capabilities, and additional numerical context features.

The two encoded representations are concatenated and provided as input to a feed-forward neural network. The network consists of two hidden layers with 64 units each and ReLU activation functions, followed by an output layer that produces scores for the available architecture actions.

The policy selects the highest-scoring valid action during deterministic selection. An exploration mechanism can also be enabled during training to select alternative valid actions.

The neural policy is trained using a one-step temporal-difference objective. Its parameters are optimized using

the Adam optimizer. This implementation provides task-conditioned action scoring and is separate from the tabular Q-learning baseline [8], [9], [11].

E. Cross-Task Neural Training and Fast Adaptation

The `meta_neural_trainer.py` module implements shared task-conditioned neural training across multiple tasks and evaluates fast adaptation on held-out tasks [9]–[11].

The task distribution is divided into training and unseen task subsets using a deterministic split. The default training ratio is 70%, with the remaining tasks reserved for unseen-task evaluation. The training and unseen task identifiers are checked to ensure that the two subsets do not overlap.

During training, a shared task-conditioned neural policy is updated using task-specific architecture states, task contexts, actions, and reward signals. The trainer records training losses and task-performance scores throughout the update process [8], [9].

For fast-adaptation evaluation, the trained policy is adapted to an unseen task using a limited number of task-specific updates. Its performance is compared with that of a freshly initialized policy receiving the same number of updates [10], [11].

The comparison investigates whether shared training across multiple tasks provides an advantage over training a new policy from scratch.

The neural training and adaptation experiments use a deterministic architecture-task compatibility evaluator. Therefore, these experiments evaluate architecture selection using a performance proxy rather than actual execution of LLM-based multi-agent tasks. The approach should be interpreted as task-conditioned cross-task training with fast-adaptation experiments, rather than as an implementation of a specific MAML, Reptile, PEARL, or RL-squared algorithm.

F. Benchmark and Evaluation Components

The `benchmark.py` module provides a benchmark suite containing 18 tasks distributed across six categories:

- **Greetings and Chit-Chat:** Simple conversational requests requiring a compact agent configuration.
- **Mathematical Reasoning:** Tasks involving mathematical calculations and reasoning.
- **Coding and Data Structures:** Programming tasks requiring code generation and implementation.
- **Research:** Tasks involving information retrieval, analysis, and synthesis.
- **Web Search:** Tasks requiring external information retrieval and time-sensitive information.
- **Cross-Domain Tasks:** Complex tasks combining multiple capabilities, such as research, coding, and verification.

The evaluation components record performance-related measurements, including token consumption, execution latency, estimated cost, capability coverage, and architecture characteristics.

The repository also includes a transfer-evaluation component for comparing task adaptation with training from scratch across multiple tasks and random seeds. These evaluation components provide the basis for analyzing architecture-selection and transfer behavior.

G. Tool Integration, Runtime, and Persistence

The system supports integration with external tools, including calculator, web search, academic search, and date/time utilities. The availability of these tools depends on the configured services and required credentials.

The runtime orchestrator manages task execution, thread context, and architecture reassessment. Dynamic workflow construction is integrated with LangGraph through the graph-related components [18], [19].

The project documentation also describes MongoDB Atlas or a local fallback for persistence, with TensorBoard and MongoDB used for telemetry and experiment tracking. The selected storage backend and logging configuration should be recorded for reproducible experimental runs.

H. API and Execution Interface

The project provides an HTTP-based service interface for interacting with the multi-agent system. The documented functionality includes health checks, chat requests, feedback submission, architecture inspection, thread management, graph inspection, metrics, and benchmark execution.

A command-line entry point is also provided for starting the application and executing supported operations. The API and runtime components connect the architecture-management, agent-execution, and evaluation modules into a unified system.

VII. Experimental Methodology and Preliminary Results

A. Benchmark Task Categories

The project benchmark documentation describes a suite of 18 tasks distributed across six categories. These categories represent different task requirements and levels of workflow complexity:

- **Greetings and Chit-Chat:** Simple conversational tasks that generally require a compact agent configuration.
- **Mathematical Reasoning:** Calculation and mathematical reasoning tasks requiring appropriate computational capabilities.
- **Coding and Data Structures:** Programming tasks involving code generation and implementation.
- **Research:** Tasks requiring information retrieval, analysis, and synthesis.
- **Web Search:** Tasks requiring external information retrieval.
- **Cross-Domain Tasks:** Complex tasks combining multiple capabilities, such as research, coding, and verification.

The benchmark contains three tasks in each category. The task definitions specify the required capabilities, difficulty, and whether research, coding, verification, or external tools are required.

B. Baseline Methods

The experimental framework considers three architecture selection approaches:

1) **Static MAS:** Uses a predefined multi-agent architecture for all incoming tasks. The agent configuration remains fixed regardless of task-specific requirements.
2) **Task-Conditioned RL:** Uses the current architecture state and task context to select architecture modification actions through a reinforcement learning policy. The repository includes tabular Q-learning and a task-conditioned neural policy as separate implementations [5]–[7], [24].
3) **Cross-Task Neural Adaptation:** Uses a shared task-conditioned neural policy trained across multiple tasks. The trained policy is evaluated on held-out tasks using a limited number of task-specific updates and is compared with a freshly initialized policy receiving the same update budget [8]–[11].

The third approach represents the repository's task-conditioned cross-task training and fast-adaptation experiment. It is not treated as a canonical MAML, Reptile, PEARL, or RL-squared implementation.

The three approaches are evaluated through different experimental procedures. The prototype dashboard observations are reported separately from the neural policy's proxy-based adaptation experiments. A direct three-way numerical comparison is not reported because the available dashboard results do not provide comparable measurements for all three approaches.

C. Evaluation Metrics

The evaluation framework considers the following performance metrics:

- **Token Usage:** Total input and output tokens consumed during task execution.
- **Execution Latency:** Elapsed time between request submission and final response generation.
- **Estimated Cost:** Cost calculated using token consumption and the applicable model-provider pricing.
- **Capability Coverage:** Proportion of required task capabilities supported by the selected architecture.
- **Response Quality:** Correctness, completeness, and relevance of the generated response.
- **Architecture Size:** Number of active agents and communication relationships in the selected architecture.
- **Rework Rate:** Frequency of correction or reassessment during task execution.

For metrics where lower values are preferable, the percentage reduction relative to a baseline is calculated as

$$\text{Reduction}(\%) = \frac{M_{\text{baseline}} - M_{\text{adaptive}}}{M_{\text{baseline}}} \times 100. \tag{17}$$

For metrics where higher values are preferable, such as capability coverage, the values are reported directly.

TABLE II
PRELIMINARY AGGREGATE VALUES DISPLAYED BY THE PROTOTYPE DASHBOARD

Metric	Static MAS	Adaptive MAS
Token Usage	3500	1808
Estimated Cost (USD)	0.00525	0.00273

D. Preliminary Prototype Dashboard Results

Table II presents the preliminary aggregate values displayed by the prototype dashboard for the Static MAS and Adaptive MAS configurations.

The displayed values indicate lower token consumption and estimated cost for the Adaptive MAS configuration compared with the Static MAS configuration. Based on the displayed aggregate values, token usage is reduced by approximately 48.34%, while estimated cost is reduced by approximately 48.00%.

The displayed latency values suggest a reduction of approximately 56.55%. However, this latency comparison requires further verification because the aggregate dashboard value does not match the arithmetic mean calculated from the category-level latency values.

These measurements are preliminary prototype dashboard observations. They should not be interpreted as independently validated results of the neural cross-task adaptation experiment.

E. Per-Category Prototype Observations

Table III presents the category-level values displayed by the prototype dashboard for the Adaptive MAS configuration.

The displayed values indicate that greetings use the lowest token count and execution latency, whereas cross-domain tasks require the highest token count, latency, and number of execution hops.

The arithmetic mean of the six displayed token values is approximately 1808.33, while the arithmetic mean of the displayed capability coverage values is approximately 99.17%. These values are consistent with the corresponding aggregate dashboard values after rounding. However, the arithmetic mean of the six category-level latency values is approximately 1983.33 ms, which differs from the aggregate dashboard value of 1825 ms. Therefore, aggregate latency is excluded from the quantitative comparison until the underlying execution logs and aggregation method have been verified.

$$\begin{aligned}\overline{L} &= \frac{1100 + 1650 + 2200 + 2200 + 1650 + 3100}{6} \\ &= 1983.33 \text{ ms}.\end{aligned} \tag{18}$$

This value differs from the aggregate dashboard latency of 1825 ms reported in Table **??**. The available displayed values do not establish the reason for this difference. Therefore, the aggregate latency must be verified against the raw execution logs and the dashboard aggregation procedure before a final latency reduction is claimed.

TABLE III
PER-CATEGORY PRELIMINARY DASHBOARD VALUES FOR ADAPTIVE MAS

Category	Tokens	Latency (ms)	Hops	Capability Coverage
Greetings	1300	1100	2	100%
Mathematics	1450	1650	3	100%
Coding	1950	2200	3	98.5%
Research	1950	2200	3	100%
Web Search	1450	1650	3	99%
Cross-Domain	2750	3100	5	97.5%

TABLE IV
SUPPLEMENTARY MOCK BENCHMARK SUMMARY

Metric	Reported Value
Number of Tasks	18
Number of Episodes	3
Baseline Token Consumption	63,000
Adapted Token Consumption	42,900
Token Reduction (%)	31.90
Baseline Estimated Cost ($)	0.0945
Adapted Estimated Cost ($)	0.0643
Cost Reduction (%)	31.96
Mean Agent Pruning Rate (%)	47.23
Mean Critical Path Length	3.00
Mean Pareto Net Utility	0.842
Capability Coverage (%)	100
Pareto-Optimal Rate (%)	100
Active RL Hit Rate (%)	61.11

F. Supplementary Mock Benchmark Summary

An additional benchmark summary was examined to analyze token consumption, estimated execution cost, and architectural efficiency across 18 tasks executed over three episodes. The summary reported a baseline token consumption of 63,000 tokens and an adapted token consumption of 42,900 tokens, corresponding to a reported reduction of 31.90%.

The estimated baseline cost was \$0.0945, while the adapted configuration recorded an estimated cost of \$0.0643, representing a reported reduction of approximately 31.96%. The benchmark also reported a mean agent pruning rate of 47.23%, a mean critical path length of 3.00, and a mean Pareto net utility of 0.842. Capability coverage and Pareto-optimal rate were both reported as 100%, while the active reinforcement-learning hit rate was 61.11%.

These values are supplementary mock-benchmark observations and are separate from the prototype dashboard results presented in Tables II and III. The detailed execution records do not independently establish paired baseline and adaptive runs. Consequently, the reported improvements should be treated as preliminary summary estimates rather than independently validated performance gains.

G. Cross-Task Training and Adaptation Evaluation

The repository implements a separate experimental procedure for evaluating task-conditioned neural training and fast adaptation [8]–[11].

The task distribution is divided into training and unseen-task subsets using a deterministic split. The default configuration uses a 70% training ratio, with the remaining tasks reserved for unseen-task evaluation. The trainer checks that training and unseen task identifiers do not overlap.

A shared task-conditioned neural policy is updated across the training tasks. Its training loss and task-performance scores are recorded during the training process.

For unseen-task evaluation, the trained policy is adapted using a limited number of task-specific updates. Its performance is compared with that of a freshly initialized policy receiving the same number of updates [9]–[11].

The adaptation improvement is defined as

$$\Delta Q_{\mathrm{adapt}} = Q_{\mathrm{after}} - Q_{\mathrm{before}}, \tag{19}$$

where Q_{before} and Q_{after} represent the task-performance scores before and after adaptation, respectively.

The advantage over training from scratch is defined as

$$\Delta Q_{\mathrm{transfer}} = Q_{\mathrm{adapted}} - Q_{\mathrm{scratch}}, \tag{20}$$

where Q_{adapted} denotes the performance after adapting the shared policy and Q_{scratch} denotes the performance of the freshly initialized policy after the same update budget.

The neural adaptation experiment uses a deterministic architecture-task compatibility evaluator. Consequently, the resulting performance scores measure architecture compatibility through a proxy and do not represent actual LLM-based multi-agent task execution.

The proxy-based adaptation results should therefore be analyzed separately from the prototype dashboard measurements. Numerical results should be included only when supported by the corresponding experiment outputs.

H. Experimental Limitations and Reproducibility

The current results provide preliminary observations of adaptive architecture selection and a separate experimental procedure for task-conditioned neural adaptation. However, the available measurements do not establish a complete controlled numerical comparison among Static MAS, Task-Conditioned RL, and Cross-Task Neural Adaptation.

A more comprehensive evaluation requires the same task instances, model provider, model version, prompt configuration, and tool availability across the compared execution-based approaches.

The neural adaptation experiment should be evaluated separately using held-out tasks, multiple random seeds, and identical adaptation budgets for the shared-policy and from-scratch configurations. Mean performance, standard deviation,

and adaptation curves should be reported when repeated experimental outputs are available.

For reproducibility, the evaluation should also record the software configuration, model identifiers, task split, random seeds, training updates, adaptation updates, and raw execution logs.

Accordingly, the dashboard observations are presented as preliminary prototype measurements, while the neural cross-task training and adaptation procedure is presented as a separate proxy-based experiment. Further controlled experiments are required to establish the effectiveness of the learned policy during actual LLM-based multi-agent execution.

VIII. Discussion and Limitations

The proposed framework investigates multi-agent architecture selection as an adaptive decision-making problem. Instead of using an identical workflow for every request, the framework allows the active agents, their assigned roles, and communication relationships to be modified according to task requirements [1]–[4]. This provides a flexible approach to constructing multi-agent workflows with different levels of complexity.

An important advantage of this design is the possibility of reducing unnecessary agent activations. Tasks that require limited capabilities can use smaller agent configurations, whereas tasks involving research, coding, or multiple reasoning stages can use additional specialized agents. This architecture-level adaptation provides a mechanism for balancing workflow complexity and execution requirements [1], [3], [17].

The structural validation mechanism further improves the reliability of architecture modification. Before a candidate configuration is converted into an executable workflow, its structural constraints are checked. These checks help prevent invalid configurations, such as disconnected execution paths or the removal of required agents. Runtime reassessment also provides a mechanism for modifying the workflow when execution feedback indicates that additional capabilities are required [2], [18], [22].

The reinforcement learning components provide a basis for learning architecture-selection decisions from task and environment feedback. The tabular Q-learning implementation learns action values associated with architecture states, while the task-conditioned neural policy incorporates task information into policy selection. The cross-task training experiments further investigate whether experience collected from multiple tasks can improve adaptation to a target task [8]–[11], [24].

A. Limitations

Despite the flexibility of the proposed framework, several limitations remain.

First, architecture selection depends on the quality of task representation. Incorrect task classification, capability identification, or complexity estimation may result in an unsuitable agent configuration. The effectiveness of adaptation therefore depends on how accurately task requirements are represented.

Second, the learning process is influenced by the diversity of the available training tasks. A limited task distribution may not provide sufficient experience for selecting suitable architectures in substantially different environments. Evaluation on additional task categories and independent held-out tasks is required to assess generalization more reliably [8]–[10].

Third, the current task-performance evaluator uses a deterministic architecture–task compatibility measure as a proxy for task performance in the adaptation experiments. This provides a controlled way to compare architecture configurations, but it does not directly measure the quality of responses generated by actual LLM agents. Consequently, improvements in proxy scores should not be interpreted as equivalent to improvements in real-world task success or response quality.

Fourth, the tabular transfer experiments and neural task-conditioned experiments use different policy representations and evaluation procedures. Their results should therefore be reported separately. A controlled comparison requires consistent task splits, adaptation budgets, evaluation metrics, and random seeds.

Fifth, the adaptation process introduces additional computational overhead. Training policies across multiple tasks and performing task-specific adaptation may require additional processing before or during execution. The potential reduction in agent invocations and resource consumption must be evaluated against this overhead [9]–[11].

Sixth, LLM-assisted exploration may introduce additional model calls, inference costs, and nondeterministic behavior. Its contribution should be isolated through ablation experiments rather than attributing all observed improvements to reinforcement learning [5]–[7].

Finally, the reward function influences the architecture selection policy. An unsuitable balance between task performance, structural validity, and architecture efficiency may encourage configurations that optimize the reward without producing reliable task outcomes. Further reward sensitivity experiments are necessary to understand this trade-off.

IX. Conclusion

This paper presented an adaptive architecture for LLM-based multi-agent systems using reinforcement learning. The framework dynamically selects active agents, assigns roles, and modifies communication links based on task requirements. Structural validation ensures that proposed architecture changes satisfy graph constraints before execution. The system also supports specialized agents, quality-based correction, and runtime reassessment. Preliminary dashboard observations indicate lower token consumption and estimated cost compared with a static configuration. These observations suggest the potential benefits of adaptive agent selection. However, further experiments are required to validate the results and establish performance improvements. Future work will evaluate the framework using diverse tasks, held-out tasks, and repeated experimental runs. Ablation studies and actual LLM execution will help measure response quality, latency, and computational

cost. Overall, this work provides a foundation for developing more efficient and reliable adaptive multi-agent systems.

A. Future Work

Future work will focus on evaluating the framework using larger task sets, repeated experiments, and ablation studies. It will also integrate actual LLM execution to measure response quality, task success, token consumption, latency, and cost. These improvements will help validate the effectiveness and reliability of the proposed adaptive architecture.

REFERENCES

[1] B. Li, Z. Zhao, D.-H. Lee, and G. Wang, "Adaptive graph pruning for multi-agent communication," in *Proceedings of the European Conference on Artificial Intelligence (ECAI 2025)*, 2025, pp. 4305–4312.

[2] E. H. Jiang, M. Li, G. Wan, X. Liang, S. Yin, Y. Wu, X. Li, Y. Sun, W. Wang, K.-W. Chang, and Y. N. Wu, "Dynamic generation of multi llm agents communication topologies with graph diffusion models," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, 2026, pp. 38 042–38 060.

[3] Z. Xiao, Q. Guo, Y. Wang, B. Wang, and H. Lu, "Agentdropout: Dynamic redundancy elimination for multi-agent collaboration efficiency," in *2026 8th International Conference on Electronics and Communication, Network and Computer Technology (ECNCT)*, 2026.

[4] M. Zhang, J. Kim, S. Xiang, J. Gao, and C. Cao, "Dynamic role assignment for multi-agent debate," *arXiv preprint arXiv:2601.17152*, 2026.

[5] S. Liu, Z. Liang, X. Lyu, and C. Amato, "Llm collaboration with multi-agent reinforcement learning," in *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, 2026.

[6] D. Qiu, D. Xu, and L. Yue, "Reinforcement learning-augmented llm agents for collaborative decision making and performance optimization," in *2025 7th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*, 2025.

[7] E. H. Jiang, L. Li, R. Sun, X. Liang, Y. Li, Y. Wu, H. Luo, H. Li, Z. Zhang, Z. Kang, K.-W. Chang, and Y. N. Wu, "Agent q-mix: Selecting the right action for llm multi-agent systems through reinforcement learning," *arXiv preprint arXiv:2604.00344*, 2026.

[8] S. Huang, Z. Zhao, Y. Zhu, and D. Zhao, "Adaptive multi-agent coordination among different team attribute tasks via contextual meta-reinforcement learning," in *2024 IEEE 13th Data Driven Control and Learning Systems Conference (DDCLS)*, Kaifeng, China, May 2024, pp. 1780–1787.

[9] J. Zhang, J. He, Y. Zhang, Y. Zang, N. Xu, and J. Cheng, "Bi-level knowledge transfer for multi-task multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems*, 2025.

[10] R. Kashikar, "Meta-reinforcement learning for emergent multi-agent languages in zero-shot coordination tasks," in *2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)*, 2025.

[11] K. Fu, Q. Liu, H. Chen, Y. Wang, J. Wang, and J. Huang, "Mixture of orthogonal experts: A novel approach to multi-agent fast policy adaptation," *Complex & Intelligent Systems*, vol. 12, no. 216, 2026.

[12] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[13] D. Lee and H. Kim, "A survey on llm agents: Architecture, applications, and challenges," in *2026 40th International Conference on Information Networking (ICOIN)*, 2026.

[14] C. Sun, S. Huang, and D. Pompili, "Llm-based multi-agent reinforcement learning: Current and future directions," *arXiv preprint arXiv:2405.11106*, 2024.

[15] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 6, 2025.

[16] Z. Jia, J. Li, X. Qu, and J. Wang, "Enhancing multi-agent systems via reinforcement learning with llm-based planner and graph-based policy," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[17] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, "Multi-agent architecture search via agentic supernet," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, 2025, pp. 75 834–75 852.

[18] J. Wang, Y. Wang, J. Li, S. Yang, K. Guo, and Y. Zhao, "Neuralfsm: Adaptive multi-agent coordination via learning finite-state execution policy," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026, pp. 33 414–33 436.

[19] Y. Dang, C. Qian, X. Luo, J. Fan, Z. Xie, R. Shi, W. Chen, C. Yang, X. Che, Y. Tian, X. Xiong, L. Han, Z. Liu, and M. Sun, "Multi-agent collaboration via evolving orchestration," in *Advances in Neural Information Processing Systems*, 2025.

[20] G. Rjoub, J. Bentahar, S. Almobydeen, W. Pedrycz, and A. Irjoob, "Adaptive agentic meta-controller (aamc): A deep reinforcement learning framework for intelligent slm/llm orchestration," *Neurocomputing*, vol. 678, p. 133192, 2026.

[21] H. Babu, P. Schillinger, and T. Asfour, "Adaptive domain modeling with language models: A multi-agent approach to task planning," in *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*, 2025.

[22] S. Wang, R. Lu, Z. Yang, Y. Wang, Y. Zhang, L. Xu, Q. Xu, G. Yin, C. Chen, and X. Guan, "Agentconductor: Topology evolution for multi-agent competition-level code generation," *arXiv preprint arXiv:2602.17100*, 2026.

[23] A. Wang and Z. Lin, "Multi-agent reinforcement learning with spatial structure awareness for topological map-based path-finding," *IEEE Robotics and Automation Letters*, vol. 11, no. 1, 2026.

[24] M.-L. Li, S. Chen, and J. Chen, "Adaptive learning: A new decentralized reinforcement learning approach for cooperative multiagent systems," *IEEE Access*, vol. 8, 2020.